AlumniConnect

S26CS6.401 – Software Engineering | Project 3 Proposal | Team 40 | Domain: Education

1. Description

University students regularly face career decisions — choosing a specialisation, preparing for placements, or breaking into a new industry — without guidance from people who have actually navigated those paths. Faculty advisors are stretched thin and rarely carry current industry exposure. Alumni hold directly relevant experience and are often willing to help, but no structured channel exists for students to find and engage them.

AlumniConnect is a web platform where students discover verified alumni, send mentorship requests, communicate in real time, and book virtual sessions, all within one interface.

Stakeholders

- **Students (Mentees):** Seek domain guidance, placement preparation, and internship advice.
- **Alumni (Mentors):** Verified graduates who register their domain expertise and availability.
- **University Admin:** Approves alumni profiles, manages the domain taxonomy, and monitors platform engagement.

2. Key Functionalities

ID	Feature	Technical Description
FR1	Alumni Directory with Ranked Matching	Queryable alumni catalogue with server-side filtering by domain, company, and availability. Ranked via a weighted scoring function over four signals: domain tag overlap, profile completeness, sessions conducted, and response rate. Scores recomputed incrementally per interaction event and persisted for fast read.
FR2	Mentorship Request System	Students submit personalised requests to available mentors. Each request transitions through an explicit state machine (Pending → Accepted | Declined | Withdrawn) enforced at the service layer. On acceptance, the Observer pattern fires a decoupled event pipeline: connection record creation, chat thread provisioning, and dual email dispatch.
FR3	Real-Time Chat	Each accepted connection opens a dedicated persistent WebSocket channel supporting text and binary PDF uploads (resumes, SOPs, portfolios), with messages durably written to the database. A pub-sub mechanism fans out messages across server instances, and a presence-tracking service maintains live online/offline state per connection.
FR4	Session Scheduling	Either party proposes a slot; the scheduling service validates it against existing bookings to prevent conflicts. On mutual confirmation, the calendar integration generates a virtual meeting link and dispatches invites. A job scheduler enqueues reminder notifications at T−24h and T−1h before each session.
FR5	Admin Dashboard	Alumni registrations enter a verification queue processed by a Chain of Responsibility pipeline: format validation, document check, and admin approval. Approved profiles become discoverable. Admins manage the domain taxonomy and view aggregated metrics: active mentors per domain, sessions held, and pending request volume.
FR6	Smart Notification Digest	Pending events per user — unread messages, requests, upcoming sessions — are aggregated by a scheduled digest pipeline rather than dispatched individually. Runs at a user-configured cadence (daily or weekly), compiling a structured per-user feed into a single batched email.
FR7	Connection Health Score	A time-decaying engagement score computed per active mentorship connection, derived from message frequency, session attendance rate, and inter-message response latency. A background scoring job recalculates scores on a rolling window; connections that breach a staleness threshold trigger automated re-engagement nudges via the notification pipeline and surface as flagged connections in the admin dashboard.

3. Technical Details

Technology Stack

Services communicate via an internal event bus; the WebSocket layer scales horizontally through a shared pub-sub channel; a persistent job queue guarantees no scheduled task is lost across restarts.

Component	Role in the System
Frontend	Single-page application; WebSocket client for real-time chat and presence.
Backend API	RESTful server and WebSocket host; routes requests across loosely coupled internal services via an event bus.
Relational Database	Persistent storage for users, connections, messages, sessions, health scores, and audit records.
In-Memory Store	Session cache, presence tracking, and pub-sub for multi-instance WebSocket fan-out.

Component	Role in the System
Job Scheduler	Persistent queue for session reminders, digest pipeline execution, and connection health score recalculation.
Authentication	Token-based auth with role-based access control (Student / Mentor / Admin) enforced at the API boundary.
Email Service	Transactional delivery for connection events, session confirmations, reminders, and digest emails.
Calendar Integration	External API invoked on session confirmation to generate and distribute virtual meeting links.

Design Patterns

Pattern	Purpose and Design Decision
Observer	Acceptance emits a domain event; independent listeners handle chat provisioning, email dispatch, and audit logging.
Strategy	Meeting provider abstracted behind a common interface; substitutable without modifying the scheduling service.
Repository	Persistence logic encapsulated behind repository interfaces; service classes remain ORM-free and unit-testable.
Facade	Mentorship Service exposes one entry point over request management, chat provisioning, and scheduling.
Chain of Responsibility	Verification passes through a handler pipeline (format check → document validation → admin approval); each stage advances or short-circuits.

Non-Functional Requirements

Category	Quantified Requirement
Performance	API response < 200 ms at p95. Chat delivery < 100 ms end-to-end. Ranked directory queries < 500 ms over 10,000 profiles.
Scalability	WebSocket layer scales horizontally via Redis pub-sub fan-out across stateless instances. Chat, notification, and job-scheduler services are independently deployable, each scalable without affecting the core API.
Availability	Uptime ≥ 99.5%. WebSocket auto-reconnects within 5 s with no message loss. Scheduled jobs persist in the queue across restarts and re-enqueue on recovery.
Reliability	No message lost on disconnect/reconnect. Concurrent slot requests serialised via optimistic locking to prevent double-booking. Critical state-mutation endpoints are idempotent.
Security	JWT tokens expire in 24 h with refresh rotation. RBAC enforced on every protected route. Alumni invisible until admin-verified. All traffic over HTTPS.
Modifiability	New meeting provider: ≤ 1 interface change. New notification channel: zero lifecycle-service changes. New ranking signal: injectable without modifying the scoring engine.

4. Expected Time to Build a Prototype

Two end-to-end flows are implemented for the prototype: the Mentorship Request Lifecycle (registration → request → acceptance → connection) and Real-Time Chat with PDF file sharing.

Phase	Duration	Focus
Research & Planning	Days 1–2	Requirement analysis, user stories, API contract definition, repository and CI setup, task assignment.
Design	Days 3–5	Database schema, API endpoint design, UI wireframes for request and chat flows, auth model finalised.
Sprint 1	Days 6–10	Auth service, student and alumni profiles, alumni directory with ranked matching.
Sprint 2	Days 11–15	Mentorship request lifecycle (send, accept, decline, connection creation) and real-time chat with text and PDF sharing. Two members on backend, two on frontend, one on integration and DevOps.
Testing	Days 16–18	Unit tests for request and chat services, integration tests for the WebSocket layer, end-to-end testing with volunteer users, bug triage.
Refinement	Days 19–21	Bug fixes, performance review, demo preparation, and project documentation.

5. Domain

Education. Where existing alumni channels — mailing lists, LinkedIn groups, annual meets — are passive and unstructured, AlumniConnect makes mentorship structured, searchable, and on-demand, directly improving placement outcomes and alumni engagement.